



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Develop a lighttpd test setup for ECH

Seán Joseph Lawlor

A Report

Presented to the University of Dublin, Trinity College
in partial fulfilment of the requirements for the degree of

BA(Mod) in Science in Computer Science

Supervisor: Stephen Farrell

May 2026

Develop a lighttpd test setup for ECH

Seán Joseph Lawlor, BA(Mod) in Science in Computer Science
University of Dublin, Trinity College, 2026

Supervisor: Stephen Farrell

This project explores the development of a local test environment for evaluating Encrypted Client Hello (ECH) using the Lighttpd web server. ECH, an extension to TLS 1.3, encrypts sensitive information in the ClientHello message to improve user privacy. The goal of this report is to build, configure, and test a working Lighttpd installation capable of performing ECH handshakes using BoringSSL and to verify its functionality through practical experimentation. The work includes compiling Lighttpd with ECH-capable TLS libraries, generating ECH configuration files, and validating results with tools such as `bssl client`, `curl`, Wireshark, and modern browsers. The outcome demonstrates the feasibility of integrating ECH within lightweight web server environments and documents configuration issues, debugging approaches, and performance observations.

Acknowledgments

Thank you Mum & Dad.

SEÁN JOSEPH LAWLOR

University of Dublin, Trinity College
May 2026

Contents

Abstract	i
Acknowledgments	ii
Chapter 1 Introduction	1
1.1 Motivation	1
1.2 Project Objectives	2
1.3 Research Contributions	4
Chapter 2 State of the Art	5
2.1 Background	5
2.1.1 Transport Layer Security 1.3	5
2.1.2 Encrypted ClientHello	5
2.1.3 TLS Libraries Supporting ECH	5
2.1.4 Lighttpd	5
2.1.5 Headless browsers	5
2.2 Closely-Related Work	5
2.3 Summary	5
2.4 Planned Testing and Evaluation	7
2.4.1 Functional Testing	7
2.4.2 Performance Testing	7
2.4.3 Browser-Based Testing	8
2.4.4 Fallback Behaviour Testing	8
2.4.5 Security and Privacy Verification	9
2.4.6 Test Automation	9
Chapter 3 Implementation	10
3.1 Planned Testing and Evaluation	10
3.1.1 Functional Testing	10
3.1.2 Performance Testing	11

3.1.3	Browser-Based Testing	11
3.1.4	Fallback Behaviour Testing	12
3.1.5	Security and Privacy Verification	12
3.1.6	Test Automation	12
3.2	Experimental Setup	13
3.2.1	Server Environment	13
3.2.2	Client Environment	13
3.2.3	Network Configuration	14
3.2.4	Test Configurations	14
3.2.5	Data Collection	14
3.3	Evaluation Metrics	14
3.3.1	TLS Handshake Time	15
3.3.2	Time to First Byte (TTFB)	15
3.3.3	Total Request Time	15
3.3.4	ECH Negotiation Success Rate	15
3.3.5	Fallback Behaviour	16
3.3.6	Visibility of Server Name Indication (SNI)	16
3.3.7	Automated ECH Test Suite	16
Chapter 4	Evaluation	18
4.1	Experiments	18
4.2	Results	19
4.3	Summary	20
4.3.1	Validation Using Automated ECH Test Suite	20
4.3.2	Negative and Robustness Testing	21
4.3.3	Manual Validation of Test Behaviour	21
Chapter 5	Results & Discussion	23
5.1	Discussion of Results	23
5.2	Limitations	24
5.3	Future Work	25
	Bibliography	26

List of Tables

4.1	Experimental matrix used for ECH behavioural validation	18
-----	---	----

List of Figures

5.1	Results:mod-openssl-ech.t	25
5.2	Results:mod-openssl-ech-extra.t	25

Chapter 1

Introduction

Encrypted ClientHello (ECH) is an emerging extension to the TLS 1.3 protocol which aims to protect sensitive metadata during the TLS handshake. While TLS encrypts application data, the initial ClientHello message remains unencrypted, revealing information such as the Server Name Indication (SNI) and Application-Layer Protocol Negotiation (ALPN). This information being shown in plain-text allows bad actors to monitor, profile, censor, or intercept users, even when TLS encryption is otherwise secure. ECH encrypts the ClientHelloInner message using Hybrid Public Key Encryption (HPKE) which conceals the intended domain and strengthens privacy at the network layer.

Lighttpd is a lightweight and high-performance open-source web server designed for environments where efficiency, low memory usage, and simplicity are essential. Unlike larger server platforms such as Apache or Nginx, Lighttpd prioritises a minimal footprint while maintaining support for modern web standards and extensible features through its modular architecture. Its event-driven design allows it to handle large volumes of concurrent connections with reduced overhead, making it suitable for embedded systems, resource-constrained deployments, and experimental research setups.

As ECH is relatively new and still undergoing standardisation, ECH is poorly tested. This project focuses on building a test environment. For ECH using a web server built with Lighttpd. The tests will include

1.1 Motivation

Metadata leakage remains a critical weakness even when content is fully encrypted. The SNI field in TLS has become a significant point of exposure, enabling censorship regimes, tracking systems, and other bad actors to monitor user activity. For organisations committed to user privacy including civil society groups, investigative journalists, and human rights organisations mitigating SNI-based surveillance is essential.

ECH offers a promising defence, but its deployment is not straightforward. Its reliance on experimental TLS libraries, DNS infrastructure, and non-standardised tooling creates high barriers for developers and researchers. There is also limited publicly available guidance documenting how ECH-enabled servers should be built, configured, and tested.

The motivation for this project is therefore twofold:

- To create a reproducible ECH testing environment suitable for experimentation, debugging, and educational use.
- To evaluate Lighttpd’s experimental ECH implementation, documenting its behaviour, limitations, and compatibility with BoringSSL’s tooling.

A reliable ECH test harness lowers the barrier to future deployment, facilitates research into evolving protocol behaviour, and supports wider adoption.

1.2 Project Objectives

The central research question guiding this work is:

How can Encrypted ClientHello be enabled, tested, and validated on a Lighttpd web server in an efficient reproducible manner?

This question is explored through the following objectives:

- **Understand the ECH protocol and surrounding technologies.** This involves studying TLS 1.3, HPKE, DNS integration requirements, and the structure of the ClientHelloInner and ClientHelloOuter messages. Practical experiments with BoringSSL are used to deepen understanding.
- **Compile and configure Lighttpd with ECH support.** Lighttpd must be rebuilt from source and linked against BoringSSL. This includes modifying build flags, resolving dependency issues, and integrating ECH key material into the server configuration.
- **Develop an automated ECH test harness.** Initial testing uses the `bssl client` debugging tool to inspect handshake behaviour. Later work incorporates automated browser-based testing with Playwright to simulate realistic client interactions.
- **Analyse behaviour through logged TLS handshakes.** Focus is placed on interpreting negotiation states, debugging errors such as `TLSV1_ALERT_ECH_REQUIRED`, and determining whether the server accepts, rejects, or attempts ECH negotiation under various configurations.

- **Incorporate automated browser-based testing.** Playwright and headless browsers are used to simulate realistic client behaviour, validate ECH negotiation in full browser stacks, and provide reproducible interaction traces for analysis.

These objectives establish the technical foundation for the implementation and analysis conducted throughout the project.

Period	Goals / Deliverables
Oct 14–27	- Begin learning LaTeX basics - Set up Ubuntu, build tools, Git
Oct 28–Nov 10	- Study ECH protocol - Test basic TLS setup and certificates - Research/testing strategy - Document initial setup in LaTeX
Nov 11–24	- Enable ECH on Lighttpd - Capture test results with <code>bscl</code> / <code>openssl</code> - Upload results to GitHub
Nov 25–Dec 8	- Test ECH using Chrome, Firefox, etc. - Introduce Playwright for automated headless testing - Validate ECH behaviour in real browsers - Update LaTeX documentation
Dec 9–22	- Basic performance measurements - Expand Playwright tests (timing, errors) - Begin structured LaTeX report
Dec 23–Jan 5	- Mid-project review and summary - Update GitHub with configs and Playwright code - Produce short progress PDF
Jan 6–19	- Refine ECH setup and tests - Improve Playwright traces, logs, screenshots - Fix server or test issues
Jan 20–Feb 2	- Finalise ECH implementation on Lighttpd - Automate recurring Playwright tests (CI-ready)
Feb 3–16	- Run comprehensive ECH tests - Compare <code>bscl client</code> vs Playwright behaviour - Draft full LaTeX report
Feb 17–Mar 1	- Review and refine LaTeX report - Add figures, tables, diagrams, browser traces
Mar 2–15	- Supervisor feedback and revisions - Finalise Playwright scripts and documentation
Mar 16–29	- Prepare presentation/demo - Test demo setup (Lighttpd + Playwright interaction)
Mar 30–Apr 12	- Submit final report and project deliverables - Stretch Goal: PR to Lighttpd - Present project

A biweekly timeline is shown below fig 1.2.1

1.3 Research Contributions

This work makes the following contributions:

- **A reproducible environment for building and testing ECH on Lighttpd.** This includes the full toolchain for compiling BoringSSL, integrating it with Lighttpd, and generating ECH key material.
- **A detailed examination of ECH handshake behaviour.** The project documents negotiation states, error modes, and observable behaviours such as `ECH_REJECTED` that arise during experimental testing.
- **Analysis of Lighttpd's experimental ECH implementation.** This includes identifying limitations, configuration pitfalls, and mismatches between BoringSSL's ECH formats and Lighttpd's expectations.
- **Platwright and headless browser.** The project establishes tests using playwright and headless browser

The methodology developed here supports future researchers studying ECH adoption, debugging server implementations, and building larger interoperability test suites.

Chapter 2

State of the Art

This chapter provides an overview of the technologies, standards, and prior work relevant to the development of an Encrypted ClientHello (ECH) test setup for Lighttpd. It first introduces the technical background required to understand ECH within the TLS 1.3 protocol and then surveys work related to ECH implementations, testing methodologies, and deployment models. The chapter concludes with a summary that compares existing approaches and highlights the gap addressed by this dissertation.

2.1 Background

2.1.1 Transport Layer Security 1.3

2.1.2 Encrypted ClientHello

2.1.3 TLS Libraries Supporting ECH

2.1.4 Lighttpd

2.1.5 Headless browsers

2.2 Closely-Related Work

2.3 Summary

This chapter introduced the technical foundations of TLS 1.3 and ECH, reviewed existing implementations of ECH across various libraries, and discussed prior work in deployment, testing, and analysis. Existing research demonstrates both the potential and challenges of deploying ECH in real-world environments.

However, very limited work has examined ECH integration on lightweight web servers such as Lighttpd. Most existing literature focuses on NGINX or general-purpose TLS libraries, leaving a gap in the evaluation of smaller, embedded-friendly servers. This dissertation addresses this gap by building a reproducible ECH testing environment specifically for Lighttpd.

2.4 Planned Testing and Evaluation

To evaluate the implementation of Encrypted ClientHello (ECH) support within the Lighttpd web server, a number of tests will be conducted. The goal of these tests is to determine whether the implementation functions correctly, maintains compatibility with existing TLS infrastructure, and to measure any performance overhead introduced by ECH.

The evaluation is divided into several categories: functional testing, performance testing, browser-based testing, fallback behaviour testing, and security verification.

2.4.1 Functional Testing

Functional testing will be used to verify that the implementation behaves as expected and that ECH is successfully negotiated between client and server.

The following aspects will be tested:

- Whether a client can successfully establish a TLS connection with the server when ECH is enabled.
- Whether the server correctly processes ECH-enabled ClientHello messages.
- Whether ECH negotiation succeeds when both the client and server support the feature.
- Whether handshake logs confirm that ECH was attempted and accepted.

These tests ensure that the core functionality of the system is working correctly before any further evaluation is performed.

2.4.2 Performance Testing

Performance testing will be conducted to determine whether enabling ECH introduces any measurable overhead during TLS connection establishment.

Two configurations will be compared:

- A baseline configuration where ECH is disabled.
- A configuration where ECH is enabled on the server.

For each configuration, a series of HTTP requests will be sent to the server and timing metrics will be recorded. These metrics include:

- TLS handshake time
- Time to first byte (TTFB)
- Total request completion time

The purpose of this testing is to evaluate whether encrypting the ClientHello message has any noticeable impact on latency or overall connection performance.

2.4.3 Browser-Based Testing

In addition to command-line testing tools, browser-based testing will be conducted to simulate real-world usage scenarios.

A web browser that supports ECH will be configured to connect to the test server. These tests will verify that:

- Web pages hosted on the Lighttpd server load correctly when ECH is enabled.
- The TLS handshake successfully negotiates ECH in a browser environment.
- No errors occur during the connection establishment process.

Automation tools may also be used to repeatedly load pages in order to observe any differences in page load performance.

2.4.4 Fallback Behaviour Testing

ECH is designed to be backwards compatible with existing TLS deployments. If a client or server cannot successfully negotiate ECH, the connection should fall back to standard TLS behaviour.

Tests will therefore be conducted to simulate scenarios where ECH cannot be used, including:

- Clients without ECH support.
- Incorrect or missing ECH configuration.
- Servers rejecting ECH keys.

In these cases, the expected behaviour is that the connection proceeds using standard TLS with plaintext Server Name Indication (SNI). This ensures compatibility with existing web infrastructure.

2.4.5 Security and Privacy Verification

One of the primary motivations behind ECH is to prevent observers from identifying the hostname being accessed during the TLS handshake.

To verify that the hostname is no longer visible in plaintext, network traffic generated during connections will be captured and analysed. The captured packets will be inspected to confirm that:

- The Server Name Indication is not visible in plaintext within the ClientHello message.
- The handshake structure reflects the use of encrypted extensions associated with ECH.

This testing ensures that the implementation successfully achieves the intended privacy improvements provided by ECH.

2.4.6 Test Automation

To improve the reliability and repeatability of the evaluation, automated scripts will be used to run tests repeatedly and collect structured output data.

Automation will allow:

- Running a large number of test requests.
- Collecting consistent timing measurements.
- Comparing results across different configurations.

This approach ensures that the performance analysis is based on a sufficiently large dataset and reduces the impact of variability between individual test runs.

Chapter 3

Implementation

3.1 Planned Testing and Evaluation

To evaluate the implementation of Encrypted ClientHello (ECH) support within the Lighttpd web server, a number of tests will be conducted. The goal of these tests is to determine whether the implementation functions correctly, maintains compatibility with existing TLS infrastructure, and to measure any performance overhead introduced by ECH.

The evaluation is divided into several categories: functional testing, performance testing, browser-based testing, fallback behaviour testing, and security verification.

3.1.1 Functional Testing

Functional testing will be used to verify that the implementation behaves as expected and that ECH is successfully negotiated between client and server.

The following aspects will be tested:

- Whether a client can successfully establish a TLS connection with the server when ECH is enabled.
- Whether the server correctly processes ECH-enabled ClientHello messages.
- Whether ECH negotiation succeeds when both the client and server support the feature.
- Whether handshake logs confirm that ECH was attempted and accepted.

These tests ensure that the core functionality of the system is working correctly before any further evaluation is performed.

3.1.2 Performance Testing

Performance testing will be conducted to determine whether enabling ECH introduces any measurable overhead during TLS connection establishment.

Two configurations will be compared:

- A baseline configuration where ECH is disabled.
- A configuration where ECH is enabled on the server.

For each configuration, a series of HTTP requests will be sent to the server and timing metrics will be recorded. These metrics include:

- TLS handshake time
- Time to first byte (TTFB)
- Total request completion time

The purpose of this testing is to evaluate whether encrypting the ClientHello message has any noticeable impact on latency or overall connection performance.

3.1.3 Browser-Based Testing

In addition to command-line testing tools, browser-based testing will be conducted to simulate real-world usage scenarios.

A web browser that supports ECH will be configured to connect to the test server. These tests will verify that:

- Web pages hosted on the Lighttpd server load correctly when ECH is enabled.
- The TLS handshake successfully negotiates ECH in a browser environment.
- No errors occur during the connection establishment process.

Automation tools may also be used to repeatedly load pages in order to observe any differences in page load performance.

3.1.4 Fallback Behaviour Testing

ECH is designed to be backwards compatible with existing TLS deployments. If a client or server cannot successfully negotiate ECH, the connection should fall back to standard TLS behaviour.

Tests will therefore be conducted to simulate scenarios where ECH cannot be used, including:

- Clients without ECH support.
- Incorrect or missing ECH configuration.
- Servers rejecting ECH keys.

In these cases, the expected behaviour is that the connection proceeds using standard TLS with plaintext Server Name Indication (SNI). This ensures compatibility with existing web infrastructure.

3.1.5 Security and Privacy Verification

One of the primary motivations behind ECH is to prevent observers from identifying the hostname being accessed during the TLS handshake.

To verify that the hostname is no longer visible in plaintext, network traffic generated during connections will be captured and analysed. The captured packets will be inspected to confirm that:

- The Server Name Indication is not visible in plaintext within the ClientHello message.
- The handshake structure reflects the use of encrypted extensions associated with ECH.

This testing ensures that the implementation successfully achieves the intended privacy improvements provided by ECH.

3.1.6 Test Automation

To improve the reliability and repeatability of the evaluation, automated scripts will be used to run tests repeatedly and collect structured output data.

Automation will allow:

- Running a large number of test requests.

- Collecting consistent timing measurements.
- Comparing results across different configurations.

This approach ensures that the performance analysis is based on a sufficiently large dataset and reduces the impact of variability between individual test runs.

3.2 Experimental Setup

This section describes the hardware, software and network configuration used to conduct the experiments evaluating Encrypted ClientHello (ECH) support in the Lighttpd web server. Defining the experimental setup ensures that the testing methodology is reproducible and that performance measurements can be interpreted in the correct context.

3.2.1 Server Environment

The test server runs a modified version of the Lighttpd web server with support for Encrypted ClientHello (ECH). The server is configured with TLS support using the BoringSSL library, which provides experimental support for ECH.

The server environment consists of:

- Operating System: Ubuntu Linux
- Web Server: Lighttpd (modified build with ECH support)
- TLS Library: BoringSSL
- Test Web Content: A simple static webpage used for repeated HTTP requests

The server is configured to serve HTTPS traffic and to publish ECH configuration information required for clients attempting ECH connections.

3.2.2 Client Environment

Testing is performed using command-line clients and browser-based clients capable of initiating TLS connections to the test server.

The client environment includes:

- Command-line testing using `curl`
- Custom TLS testing using BoringSSL client utilities

- Browser-based testing using a modern web browser supporting ECH

Command-line tools allow for controlled and repeatable testing of TLS handshakes, while browser-based tests simulate realistic user behaviour when accessing websites over HTTPS.

3.2.3 Network Configuration

The testing environment is configured to minimise external network variability. In most tests the client and server operate within the same local network environment.

This approach ensures that measured performance differences are primarily caused by the TLS configuration rather than network latency or external routing conditions.

Where required, packet capture tools are used to observe TLS handshakes and confirm whether ECH negotiation occurs successfully.

3.2.4 Test Configurations

Two primary configurations are used during testing in order to evaluate the impact of ECH.

- **Baseline Configuration:** Standard TLS handshake with ECH disabled.
- **ECH Configuration:** TLS handshake with ECH enabled and supported by both the client and server.

By comparing results from these configurations it is possible to determine whether enabling ECH introduces any performance overhead or behavioural differences.

3.2.5 Data Collection

Test results are collected through automated scripts which repeatedly initiate HTTPS connections to the server. Timing information and connection logs are recorded for later analysis.

The collected data will be used to compare connection performance and verify that ECH negotiation behaves as expected across different test scenarios.

3.3 Evaluation Metrics

In order to evaluate the behaviour and performance impact of Encrypted ClientHello (ECH), a number of metrics will be collected during testing. These metrics allow the

implementation to be compared with a standard TLS configuration where ECH is disabled. The goal is to determine whether ECH introduces any measurable overhead while also verifying that the protocol behaves as expected.

3.3.1 TLS Handshake Time

One of the most important metrics in this evaluation is the time required to complete the TLS handshake. The TLS handshake is responsible for establishing a secure connection between the client and server before any application data is transmitted.

When ECH is enabled, additional cryptographic operations are performed to encrypt parts of the ClientHello message. Measuring handshake time allows the evaluation to determine whether these additional operations introduce any noticeable latency.

This metric will be collected by recording the time taken to establish a successful HTTPS connection during repeated test requests.

3.3.2 Time to First Byte (TTFB)

Time to First Byte (TTFB) measures the time between initiating an HTTP request and receiving the first byte of the server's response. This metric reflects the combined impact of network latency, TLS negotiation, and server processing time.

By comparing TTFB measurements between the baseline TLS configuration and the ECH-enabled configuration, it is possible to observe whether enabling ECH has any effect on perceived response time.

3.3.3 Total Request Time

Total request time measures the full duration from initiating the request until the entire response has been received by the client.

This metric provides a broader view of performance and helps determine whether ECH impacts overall request completion time when retrieving web content from the server.

3.3.4 ECH Negotiation Success Rate

Another important metric is whether ECH negotiation is successfully completed between the client and server. During testing, connection logs will be examined to determine:

- Whether the client attempted to use ECH
- Whether the server accepted the ECH configuration

- Whether the connection successfully completed using ECH

Tracking the success rate of ECH negotiation ensures that the implementation behaves correctly and consistently during repeated test runs.

3.3.5 Fallback Behaviour

ECH is designed to maintain compatibility with existing TLS infrastructure by allowing connections to fall back to standard TLS when ECH cannot be negotiated.

During testing, scenarios will be created where ECH negotiation fails or is unsupported. In these cases, the expected behaviour is that the connection proceeds using a standard TLS handshake.

Observing this behaviour ensures that the system remains compatible with clients and servers that do not yet support ECH.

3.3.6 Visibility of Server Name Indication (SNI)

One of the primary privacy goals of ECH is to prevent observers from identifying the hostname being accessed during the TLS handshake.

Packet captures will therefore be inspected to determine whether the Server Name Indication (SNI) remains visible in plaintext during connections. When ECH is functioning correctly, the hostname should be encrypted and not directly observable within the ClientHello message.

Verifying this behaviour confirms that the implementation achieves the intended privacy improvements.

3.3.7 Automated ECH Test Suite

In addition to the planned testing categories, a dedicated automated test suite was implemented using Lighttpd's native Perl-based testing framework. This allowed for precise and repeatable validation of ECH behaviour at both the TLS protocol level and the application level.

The test suite integrates command-line tools such as `curl` and BoringSSL's `bssl client` to validate both HTTP responses and TLS handshake properties.

The automated tests cover the following scenarios:

- Successful TLS connections without ECH, verifying compatibility with standard clients.

- Successful TLS connections with ECH enabled, confirming correct negotiation and acceptance of ECH.
- Verification of ECH negotiation status using client-side handshake output (e.g. `Encrypted ClientHello: yes/no`).
- Rejection of invalid or mismatched ECH configurations, ensuring incorrect inputs are not accepted.
- Validation that the server remains stable and continues to serve HTTPS traffic after failed ECH attempts.

These tests ensure that ECH functionality is not only implemented correctly but also behaves consistently across multiple connection attempts.

Chapter 4

Evaluation

The Evaluation chapter should present a comparison of the work that forms the basis of the dissertation and existing work. At a higher level, it should demonstrate an awareness of the relationship of the dissertation work to the research area that it is based in.

4.1 Experiments

To evaluate the correctness and behavioural properties of the implemented ECH test harness, a full $2 \times 2 \times 2$ experimental matrix was executed. The independent variables were:

- **Server Mode:** ECH enabled (`ech_on`) or ECH disabled (`ech_off`)
- **Client Mode:** ECH-capable client (`ech`) or non-ECH client (`no_ech`)
- **HTTP Mode:** HTTP/1.1 or HTTP/2

This produced eight experimental configurations. Each configuration was executed automatically using the custom `run_matrix.sh` harness.

Server Mode	Client Mode	HTTP Mode	Expected Behaviour
ech_on	ech	http1/http2	ECH Accepted
ech_on	no_ech	http1/http2	ECH Fallback
ech_off	ech	http1/http2	ECH Rejected
ech_off	no_ech	http1/http2	Normal TLS

Table 4.1: Experimental matrix used for ECH behavioural validation

For each run, the following metrics were recorded:

- TLS handshake outcome (accepted, fallback, failed)
- TLS handshake duration (ms)
- HTTP status code
- HTTP protocol negotiation (1.1 or 2)
- Total request time (seconds)

4.2 Results

All eight configurations executed successfully at the HTTP layer. The server started correctly in all cases (0 startup failures). All requests returned HTTP 200, and both HTTP/1.1 and HTTP/2 negotiation succeeded in all runs.

ECH Behavioural Outcomes

- ECH Accepted: 2 runs (ech_on + ech client)
- ECH Fallback: 4 runs (all no_ech client runs)
- ECH Rejected: 2 runs (ech_off + ech client)

The two rejection cases correspond exactly to scenarios where the client attempted ECH while the server had ECH disabled. In these cases, the BoringSSL client reported ECH_REJECTED, confirming correct enforcement of server configuration.

Handshake and Request Timing

Representative measurements (in milliseconds) are shown below:

Configuration	Handshake (ms)	Total Time (s)
ech_on + ech + http1	77	0.0097
ech_on + no_ech + http1	37	0.0104
ech_off + no_ech + http1	31	0.0164

Across all runs:

- HTTP success rate: 8/8 (100%)
- Protocol negotiation success: 8/8 (100%)

- Server stability: 100%

The results confirm correct ECH negotiation behaviour without negatively impacting HTTP functionality.

4.3 Summary

This evaluation demonstrates that the implemented test harness correctly distinguishes between ECH acceptance, fallback, and rejection scenarios across both HTTP/1.1 and HTTP/2.

All configurations executed without server instability, and application-layer communication remained fully functional. The results validate both the correctness of the ECH configuration and the reliability of the automated evaluation framework.

These findings establish a stable foundation for subsequent performance analysis and further experimentation involving real-world discovery mechanisms or browser-based clients.

4.3.1 Validation Using Automated ECH Test Suite

In addition to the matrix-based experiments, a dedicated automated test suite was used to validate the correctness and robustness of the ECH implementation.

The test suite was implemented using Lighttpd's native Perl testing framework and extended with ECH-specific test cases. These tests exercised a broader range of behaviours than the experimental matrix, including protocol correctness, application-level routing, and error handling.

At the protocol level, the tests verified that:

- Non-ECH clients complete standard TLS handshakes.
- ECH-capable clients successfully negotiate ECH when the server is configured correctly.
- Invalid or mismatched ECH configurations are rejected, with the client reporting `ECH_REJECTED`.

At the application level, a specialised configuration using separate public and hidden virtual hosts was used to validate observable behaviour. These tests demonstrated that:

- Requests for a hidden hostname without ECH fall back to the public virtual host.
- Requests with valid ECH configuration are routed to the hidden virtual host.

- Invalid ECH inputs do not expose hidden content.

This confirms that ECH acceptance directly influences server-side routing behaviour, rather than only affecting TLS handshake metadata.

4.3.2 Negative and Robustness Testing

The automated test suite also included a range of negative test cases designed to evaluate system robustness under incorrect configurations.

These tests included:

- Missing ECH key directories.
- Malformed ECH key files.
- Truncated ECH configuration data.

In particular, malformed ECH configurations were generated by truncating valid configuration data by a single byte. Despite being structurally similar to valid inputs, these configurations were correctly rejected by the server.

Across all negative test cases:

- ECH negotiation failed cleanly without hanging.
- Appropriate error messages were logged.
- The server continued to function normally for standard HTTPS requests.

This demonstrates that the implementation is robust and does not introduce instability when handling invalid or unexpected inputs.

4.3.3 Manual Validation of Test Behaviour

To ensure that the automated tests were not acting as a black-box verification of script execution, key behaviours were reproduced manually using command-line tools.

Using `curl` and `bssl client`, it was confirmed that:

- Non-ECH connections to the hidden hostname return public fallback content.
- ECH-enabled connections return hidden virtual host content.
- Invalid ECH configurations result in rejection and do not return HTTP 200 responses.

Additionally, test fixtures were deliberately modified (e.g. swapping public and hidden content) to confirm that the automated tests fail when expected. This provides strong evidence that the test suite validates real system behaviour rather than superficial execution success.

Chapter 5

Results & Discussion

This chapter summarises the key findings of the evaluation and discusses the implications of the results in the context of Encrypted ClientHello (ECH) and secure web communication.

5.1 Discussion of Results

The evaluation demonstrated that the implemented ECH test harness correctly distinguishes between ECH acceptance, fallback, and rejection scenarios across all tested configurations. The experimental matrix confirmed that behaviour is consistent with the expected TLS semantics: ECH is accepted only when both client and server support it, rejected when the server is not configured for ECH, and otherwise falls back to standard TLS.

A key finding is that enabling ECH does not negatively impact standard HTTPS functionality. All configurations successfully completed TLS handshakes and returned valid HTTP responses, indicating that ECH can be integrated without breaking compatibility with existing clients or protocols. This aligns with the design goal of ECH as a backwards-compatible extension to TLS.

In addition to the experimental matrix, a significant contribution of this work is the development of an automated ECH test suite integrated into Lighttpd’s native testing framework. This suite extends beyond simple configuration testing by validating protocol-level behaviour, application-level routing, and robustness under invalid inputs. Unlike the matrix experiments, which evaluate predefined scenarios, the test suite provides repeatable and fine-grained verification of ECH behaviour across a wider range of conditions, including malformed configurations and sequential connection consistency.

From a performance perspective, the results show a modest increase in handshake time when ECH is enabled. This overhead is expected due to the additional crypto-

graphic operations required to encrypt the ClientHello message. However, the impact on total request time was minimal, suggesting that ECH can be deployed without significant degradation in user-perceived performance.

A particularly important result comes from the behavioural testing using separate public and hidden virtual hosts. These tests demonstrated that ECH acceptance enables access to hidden content, while non-ECH clients are restricted to public fallback content. This confirms that ECH affects observable server behaviour, rather than only influencing TLS handshake metadata.

This result has broader implications. It suggests that ECH can be used not only as a privacy mechanism to hide the Server Name Indication (SNI), but also as a mechanism for conditional service exposure. In such a model, sensitive services could be made accessible only to clients capable of negotiating ECH, effectively providing an additional layer of access control at the transport layer.

The robustness tests, implemented as part of the automated test suite, further strengthen the evaluation. These tests included malformed and truncated ECH configurations, missing key material, and invalid client inputs. In all cases, ECH negotiation was correctly rejected without causing server instability. The server remained fully operational for standard HTTPS requests, demonstrating that the test suite verifies not only correctness but also safe handling of erroneous inputs.

In addition, manual validation confirmed that the automated test suite reflects real protocol behaviour. By reproducing key scenarios using command-line tools and deliberately modifying test fixtures, it was verified that the tests detect genuine changes in system behaviour. This increases confidence in the reliability of the evaluation framework.

Overall, the results validate not only the correctness of the ECH implementation but also the effectiveness of the developed test suite as a reliable framework for evaluating ECH behaviour in web servers.

5.2 Limitations

Despite the successful evaluation, several limitations should be acknowledged.

First, the testing environment was limited to a controlled local setup using a modified Lighttpd server and BoringSSL client tools. While this provides precise control over experimental conditions, it does not fully represent real-world deployment environments.

Second, browser-based support for ECH remains limited and experimental. As a result, large-scale testing with real user traffic and diverse client implementations was not possible within the scope of this project.

Third, the evaluation focused primarily on correctness and behavioural validation, with

```

sean@sensBell:~/final_year_project/lighttpd/test$ ./mod-openssl-ech.t
1..34
ok 1 - generated TLS certificate fixture
ok 2 - generated server ECH fixture
ok 3 - generated mismatched client ECH fixture
ok 4 - generated malformed ECH fixture
ok 5 - Starting lighttpd with mod-openssl-ech.conf
ok 6 - ECH-enabled server serves HTTPS
ok 7 - non-ECH client completed TLS handshake
ok 8 - non-ECH client stayed on standard TLS
ok 9 - non-ECH client received HTTP 200
ok 10 - ECH client completed TLS handshake
ok 11 - valid ECH was accepted
ok 12 - ECH client received HTTP 200
ok 13 - mismatched ECH input was rejected without hanging
ok 14 - mismatched ECH input produced ECH rejection
ok 15 - server stayed healthy after mismatched ECH input
ok 16 - Stopping lighttpd
ok 17 - Starting lighttpd with mod-openssl-tls.conf
ok 18 - non-ECH server serves HTTPS
ok 19 - ECH-capable client was rejected when ECH was disabled
ok 20 - ECH-disabled server returned ECH rejection
ok 21 - non-ECH server stayed healthy after ECH attempt
ok 22 - Stopping lighttpd
ok 23 - starting lighttpd with missing ECH keydir succeeds
ok 24 - missing ECH keydir rejected client without hanging
ok 25 - missing ECH keydir produced ECH rejection
ok 26 - missing ECH keydir was logged
ok 27 - server stayed healthy with missing ECH keydir
ok 28 - Stopping lighttpd
ok 29 - starting lighttpd with malformed ECH key file succeeds
ok 30 - malformed ECH key file rejected client without hanging
ok 31 - malformed ECH key file produced ECH rejection
ok 32 - malformed ECH key file was logged
ok 33 - server stayed healthy with malformed ECH key file
ok 34 - Stopping lighttpd

```

Figure 5.1: Results:mod-openssl-ech.t

```

sean@sensBell:~/final_year_project/lighttpd/test$ ./mod-openssl-ech-extra.t
1..30
ok 1 - generated TLS certificate fixture
ok 2 - generated server ECH fixture
ok 3 - generated malformed client ECH config fixture
ok 4 - wrote distinct public and hidden vhost content fixtures
ok 5 - Starting lighttpd with mod-openssl-ech-extra.conf
ok 6 - public vhost baseline served expected content without ECH
ok 7 - ECH fallback: non-ECH client completed TLS handshake
ok 8 - ECH fallback: non-ECH handshake stayed on standard TLS
ok 9 - ECH fallback: non-ECH request returned HTTP 200
ok 10 - ECH fallback: hidden host without ECH served public vhost content
ok 11 - ECH-only host: non-ECH request did not expose hidden content
ok 12 - ECH-only host: ECH client completed TLS handshake
ok 13 - ECH-only host: ECH was accepted
ok 14 - ECH-only host: ECH request returned HTTP 200
ok 15 - ECH-only host: ECH request served hidden vhost content
ok 16 - invalid ECH config was rejected without hanging
ok 17 - invalid ECH config did not complete HTTP 200
ok 18 - invalid ECH config produced a failure trace
ok 19 - server stayed healthy after invalid ECH input
ok 20 - TLS vs ECH comparison: handshake traces differ as expected
ok 21 - sequential ECH connection \#1 stayed consistent
ok 22 - sequential ECH connection \#2 stayed consistent
ok 23 - sequential ECH connection \#3 stayed consistent
ok 24 - sequential ECH connection \#4 stayed consistent
ok 25 - sequential ECH connection \#5 stayed consistent
ok 26 - HTTP/1.1 ECH request completed TLS handshake
ok 27 - HTTP/1.1 ECH request kept ECH enabled
ok 28 - HTTP/1.1 ECH request returned HTTP 200
ok 29 - HTTP/1.1 ECH request served hidden vhost content
ok 30 - Stopping lighttpd

```

Figure 5.2: Results:mod-openssl-ech-extra.t

only limited performance analysis. While preliminary results suggest minimal overhead, more comprehensive benchmarking would be required to draw definitive conclusions about performance at scale.

Finally, the implementation relies on manually generated ECH configurations rather than automated discovery mechanisms such as DNS-based ECH configuration distribution. This limits the realism of the deployment scenario.

5.3 Future Work

Bibliography